JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Class LogManager

java.lang.Object
    java.util.logging.LogManager

```
public class LogManager
extends Object
```

There is a single global LogManager object that is used to maintain a set of shared state about Loggers and log services.

This LogManager object:

- Manages a hierarchical namespace of Logger objects. All named Loggers are stored in this namespace.
- Manages a set of logging control properties. These are simple key-value pairs that can be used by Handlers and other logging objects to configure themselves.

The global LogManager object can be retrieved using LogManager.getLogManager(). The LogManager object is created during class initialization and cannot subsequently be changed.

At startup the LogManager class is located using the java.util.logging.manager system property.

The LogManager defines two optional system properties that allow control over the initial configuration:

- "java.util.logging.config.class"
- "java.util.logging.config.file"

These two properties may be specified on the command line to the "java" command, or as system property definitions passed to JNI_CreateJavaVM.

If the "java.util.logging.config.class" property is set, then the property value is treated as a class name. The given class will be loaded, an object will be instantiated, and that object's constructor is responsible for reading in the initial configuration. (That object may use other system properties to control its configuration.) The alternate configuration class can use `readConfiguration(InputStream)` to define properties in the LogManager.

If "java.util.logging.config.class" property is **not** set, then the "java.util.logging.config.file" system property can be used to specify a properties file (in java.util.Properties format). The initial logging configuration will be read from this file.

If neither of these properties is defined then the LogManager uses its default configuration. The default configuration is typically loaded from the properties file "`lib/logging.properties`" in the Java installation directory.

The properties for loggers and Handlers will have names starting with the dot-separated name for the handler or logger.

The global logging properties may include:

- A property "handlers". This defines a whitespace or comma separated list of class names for handler classes to load and register as handlers on the root Logger (the Logger named ""). Each class name must be for a Handler class which has a default constructor. Note that these Handlers may be created lazily, when they are first used.
- A property "<logger>.handlers". This defines a whitespace or comma separated list of class names for handlers classes to load and register as handlers to the specified logger. Each class name must be for a Handler class which has a default constructor. Note that these Handlers may be created lazily, when they are first used.
- A property "<logger>.useParentHandlers". This defines a boolean value. By default every logger calls its parent in addition to handling the logging message itself, this often result in messages being handled by the root logger as well. When setting this property to false a Handler needs to be configured for this logger otherwise no logging messages are delivered.
- A property "config". This property is intended to allow arbitrary configuration code to be run. The property defines a whitespace or comma separated list of class names. A new instance will be created for each named class. The default constructor of each class may execute arbitrary code to update the logging configuration, such as setting logger levels, adding handlers, adding filters, etc.

Note that all classes loaded during LogManager configuration are first searched on the system class path before any user class path. That includes the LogManager class, any config classes, and any handler classes.

Loggers are organized into a naming hierarchy based on their dot separated names. Thus "a.b.c" is a child of "a.b", but "a.b1" and a.b2" are peers.

All properties whose names end with ".level" are assumed to define log levels for Loggers. Thus "foo.level" defines a log level for the logger called "foo" and (recursively) for any of its children in the naming hierarchy. Log Levels are applied in the order they are defined in the properties file. Thus level settings for child nodes in the tree should come after settings for their parents. The property name ".level" can be used to set the level for the root of the tree.

All methods on the LogManager object are multi-thread safe.

Since:

1.4

Field Summary

Fields

Modifier and Type	Field	Description
static String	LOGGING_MXBEAN_NAME	String representation of the ObjectName for the management interface for the logging facility.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	LogManager()	Protected constructor.

Method Summary

Deprecat

Modifier and Type	Method	Des
boolean	addLogger(Logger logger)	Add
void	addPropertyChangeListener(PropertyChangeListener l)	Dep The Pro crea imp mod Java

		met in a glob dete logg over **rea** met
void	**checkAccess**()	Che cont mod conf
Logger	**getLogger**(**String** name)	Met logg
Enumeration<**String**>	**getLoggerNames**()	Get kno
static **LoggingMXBean**	**getLoggingMXBean**()	Retu man
static **LogManager**	**getLogManager**()	Retu Log
String	**getProperty**(**String** name)	Get prop
void	**readConfiguration**()	Rein prop logg
void	**readConfiguration**(**InputStream** ins)	Rein prop logg the shou java
void	**removePropertyChangeListener**(**PropertyChangeListener** l)	**Dep** The Prop crea imp mod Java met in a glob

		dete logg over **rea** met
void	**reset**()	Res con

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

LOGGING_MXBEAN_NAME

```
public static final String LOGGING_MXBEAN_NAME
```

String representation of the ObjectName for the management interface for the logging facility.

Since:

1.5

See Also:

PlatformLoggingMXBean, LoggingMXBean, Constant Field Values

Constructor Detail

LogManager

```
protected LogManager()
```

Protected constructor. This is protected so that container applications (such as J2EE containers) can subclass the object. It is non-public as it is intended that there only be one LogManager object, whose value is retrieved by calling LogManager.getLogManager.

Method Detail

getLogManager

```
public static LogManager getLogManager()
```

Returns the global LogManager object.

Returns:

the global LogManager object

addPropertyChangeListener

```
@Deprecated
public void addPropertyChangeListener(PropertyChangeListener l)
                                       throws SecurityException
```

Deprecated. *The dependency on `PropertyChangeListener` creates a significant impediment to future modularization of the Java platform. This method will be removed in a future release. The global `LogManager` can detect changes to the logging configuration by overridding the `readConfiguration` method.*

Adds an event listener to be invoked when the logging properties are re-read. Adding multiple instances of the same event Listener results in multiple entries in the property event listener table.

WARNING: This method is omitted from this class in all subset Profiles of Java SE that do not include the `java.beans` package.

Parameters:

`l` - event listener

Throws:

`SecurityException` - if a security manager exists and if the caller does not have LoggingPermission("control").

`NullPointerException` - if the PropertyChangeListener is null.

removePropertyChangeListener

```
@Deprecated
public void removePropertyChangeListener(PropertyChangeListener l)
                                          throws SecurityException
```

Deprecated. *The dependency on `PropertyChangeListener` creates a significant impediment to future modularization of the Java platform. This method will be removed in a future release. The global `LogManager` can*

detect changes to the logging configuration by overridding the `readConfiguration` method.

Removes an event listener for property change events. If the same listener instance has been added to the listener table through multiple invocations of `addPropertyChangeListener`, then an equivalent number of `removePropertyChangeListener` invocations are required to remove all instances of that listener from the listener table.

Returns silently if the given listener is not found.

WARNING: This method is omitted from this class in all subset Profiles of Java SE that do not include the `java.beans` package.

Parameters:

`l - event listener (can be null)`

Throws:

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

addLogger

```
public boolean addLogger(Logger logger)
```

Add a named logger. This does nothing and returns false if a logger with the same name is already registered.

The Logger factory methods call this method to register each newly created Logger.

The application should retain its own reference to the Logger object to avoid it being garbage collected. The LogManager may only retain a weak reference.

Parameters:

`logger - the new logger.`

Returns:

`true if the argument logger was registered successfully, false if a logger of that name already exists.`

Throws:

`NullPointerException - if the logger name is null.`

getLogger

```
public Logger getLogger(String name)
```

Method to find a named logger.

Note that since untrusted code may create loggers with arbitrary names this method should not be relied on to find Loggers for security sensitive logging. It is also important to note that the Logger associated with the String `name` may be garbage collected at any time if there is no strong reference to the Logger. The caller of this method must check the return value for null in order to properly handle the case where the Logger has been garbage collected.

Parameters:

`name - name of the logger`

Returns:

`matching logger or null if none is found`

getLoggerNames

```
public Enumeration<String> getLoggerNames()
```

Get an enumeration of known logger names.

Note: Loggers may be added dynamically as new classes are loaded. This method only reports on the loggers that are currently registered. It is also important to note that this method only returns the name of a Logger, not a strong reference to the Logger itself. The returned String does nothing to prevent the Logger from being garbage collected. In particular, if the returned name is passed to `LogManager.getLogger()`, then the caller must check the return value from `LogManager.getLogger()` for null to properly handle the case where the Logger has been garbage collected in the time since its name was returned by this method.

Returns:

`enumeration of logger name strings`

readConfiguration

```
public void readConfiguration()
                       throws IOException,
                              SecurityException
```

Reinitialize the logging properties and reread the logging configuration.

The same rules are used for locating the configuration properties as are used at startup. So normally the logging properties will be re-read from the same file that was used at startup.

Any log level definitions in the new configuration file will be applied using Logger.setLevel(), if the target Logger exists.

A PropertyChangeEvent will be fired after the properties are read.

Throws:

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").

IOException - if there are IO problems reading the configuration.

reset

```
public void reset()
           throws SecurityException
```

Reset the logging configuration.

For all named loggers, the reset operation removes and closes all Handlers and (except for the root logger) sets the level to null. The root logger's level is set to Level.INFO.

Throws:

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").

readConfiguration

```
public void readConfiguration(InputStream ins)
                       throws IOException,
                              SecurityException
```

Reinitialize the logging properties and reread the logging configuration from the given stream, which should be in java.util.Properties format. A PropertyChangeEvent will be fired after the properties are read.

Any log level definitions in the new configuration file will be applied using Logger.setLevel(), if the target Logger exists.

Parameters:

ins - stream to read properties from

Throws:

SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").

IOException - if there are problems reading from the stream.

getProperty

```
public String getProperty(String name)
```

Get the value of a logging property. The method returns null if the property is not found.

Parameters:

`name - property name`

Returns:

`property value`

checkAccess

```
public void checkAccess()
                 throws SecurityException
```

Check that the current context is trusted to modify the logging configuration. This requires LoggingPermission("control").

If the check fails we throw a SecurityException, otherwise we return normally.

Throws:

`SecurityException - if a security manager exists and if the caller does not have LoggingPermission("control").`

getLoggingMXBean

```
public static LoggingMXBean getLoggingMXBean()
```

Returns `LoggingMXBean` for managing loggers. An alternative way to manage loggers is through the `PlatformLoggingMXBean` interface that can be obtained by calling:

```
PlatformLoggingMXBean logging = ManagementFactory.getPlatformMXBean(Platform
```

Returns:

`a LoggingMXBean object.`

Since:

`1.5`

See Also:

`PlatformLoggingMXBean`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.